

Applications of QUIC

Server-Side Connection Migration

IPFS, DNS+Anycast, Load Balancing, Health-Checked Migration, and BitTorrent

Research Application Analysis • July 2026

This document analyzes five potential applications of QUIC server-side connection migration (RFC 9000, Section 9.6): (1) integration with IPFS for seamless peer failover, (2) DNS+Anycast hybrid routing with post-connection optimization, (3) QUIC-native load balancing at L3/L4/L7 layers, (4) health-checked migration with pre-validation of preferred servers, and (5) BitTorrent peer migration for seeder failover and WebSeed load distribution. For each application, we provide technical feasibility analysis, identify challenges, and propose proof-of-concept experiments that can be conducted on our existing four-machine testbed. We conclude that **health-checked load balancing** is the most immediately viable application, while **IPFS integration** and **BitTorrent peer migration** present the most novel research contributions.

Contents

1. Background: Our Server-Side Migration Implementation
2. Application 1: IPFS + QUIC Server Migration
 - 2.1 How IPFS Uses QUIC Today
 - 2.2 Can IPFS Use Server-Side Migration?
 - 2.3 Technical Feasibility Analysis
 - 2.4 Challenges and Limitations
 - 2.5 Proposed Experiment
3. Application 2: DNS + Anycast Hybrid Routing
 - 3.1 The Anycast Problem with Stateful Protocols
 - 3.2 How Server Migration Solves It
 - 3.3 Proposed Experiment
4. Application 3: QUIC-Native Load Balancing (L3/L4/L7)
 - 4.1 Traditional Load Balancing Layers
 - 4.2 QUIC Migration as a Load Balancer
 - 4.3 Comparison with Existing Approaches
 - 4.4 Proposed Experiment
5. Application 4: Health-Checked Migration
 - 5.1 The Problem: Migrating to a Dead Server
 - 5.2 Health Check Design
 - 5.3 Implementation Plan
 - 5.4 Proposed Experiment
6. Application 5: BitTorrent Peer Migration
 - 6.1 BitTorrent Architecture and Transport
 - 6.2 How QUIC Server Migration Applies
 - 6.3 Three Migration Scenarios
 - 6.4 Comparison with IPFS Migration
 - 6.5 Challenges
 - 6.6 Proposed Experiment
7. Comparative Analysis of All Applications
8. Recommended Research Roadmap
9. References

1. Background: Our Server-Side Migration Implementation

We have a working cross-machine QUIC server-side migration implementation using Mozilla's Neqo (Rust). The system operates on a four-machine LAN testbed:

Role	Machine	IP Address	Function
Client	optiplex7010	141.217.168.127	Firefox browser (HTTP/3)
Primary Server	opti7040	141.217.168.152	Completes TLS handshake, exports state
Preferred Server	homeserver2	141.217.168.143	Imports state, handles PATH_CHALLENGE
Redis Server	Proxmox VM	141.217.168.200	State transfer coordination

The migration transfers 445 bytes of state (TLS secrets, Connection IDs, client address, packet numbers) from the primary to the preferred server. We support five state transfer backends: TCP Push, HTTP Pull, Redis KV, Redis Pub/Sub, and File. The system has been verified with Firefox 151.0.3 performing real HTTP/3 page loads with transparent migration.

2. Application 1: IPFS + QUIC Server Migration

2.1 How IPFS Uses QUIC Today

The InterPlanetary File System (IPFS) is a peer-to-peer content-addressed storage network. Since 2022, QUIC has become the **default transport** in IPFS via the libp2p networking stack. Key facts about IPFS's QUIC usage:

- **libp2p transport:** IPFS uses libp2p, which supports QUIC via the go-libp2p-quic-transport module (using quic-go) or rust-libp2p (using quinn). QUIC replaced TCP+Noise as the default transport in Kubo v0.18+ (2023).
- **Peer identity:** Every IPFS node has a PeerID derived from its cryptographic public key. libp2p uses TLS 1.3 over QUIC with peer authentication via the libp2p TLS extension (inserting the peer's public key into the TLS certificate).
- **Content routing:** Content is identified by CIDs (Content Identifiers). Clients discover which peers hold content via the Kademlia DHT, IPNI (InterPlanetary Network Indexer), or Bitswap protocol.
- **Multiple providers:** The same CID can be served by **many different peers**. This is fundamental to IPFS's design -- content is replicated across the network.
- **Connection multiplexing:** A single QUIC connection to a peer can carry multiple streams (Bitwap blocks, DHT queries, pubsub messages). Losing a connection means losing all in-progress streams to that peer.

2.2 Can IPFS Use Server-Side Migration? -- Verdict: YES, with caveats

The question is whether an IPFS node (acting as a content server) can use QUIC's preferred_address mechanism to redirect a client to a different IPFS node that also holds the requested content. We analyze this along three dimensions:

Scenario A: IPFS Gateway Migration (Most Feasible)

IPFS gateways (e.g., ipfs.io, dweb.link, Cloudflare IPFS) are centralized HTTP frontends that serve IPFS content to regular web browsers. These gateways are increasingly serving content over HTTP/3 (QUIC). In this scenario:

- A gateway cluster operates multiple backend nodes, all of which can resolve any CID.
- The primary gateway completes the QUIC handshake and resolves the client's CID request.
- If the primary is overloaded, it migrates the connection to a less-loaded gateway backend.
- The preferred gateway imports the TLS state and continues serving the content.
- This is **directly analogous** to our current implementation -- the IPFS gateway acts as the primary server, and another gateway instance acts as the preferred server.

Feasibility: HIGH. IPFS gateways are controlled infrastructure with known topology. Content availability is guaranteed (gateways can fetch any CID). This is essentially load balancing for IPFS gateways using QUIC migration instead of traditional L4/L7 LBs.

Scenario B: Peer-to-Peer Content Provider Migration (Novel but Challenging)

In pure peer-to-peer IPFS, a node serving content could migrate the client to another peer that also holds the same CID. This is the most novel application:

- **Content-addressed advantage:** Because IPFS uses CIDs, the client can verify that the content received from the new peer is identical to what it would have received from the original peer. Data integrity is guaranteed regardless of which peer serves it.
- **Provider discovery:** The serving peer can query the DHT to find other providers of the same CID and select one as the preferred_address target.
- **Peer identity challenge:** libp2p binds QUIC connections to PeerIDs via TLS certificates. Migrating to a different peer means the TLS identity changes. The libp2p TLS extension authenticates the peer's identity during the handshake -- a migration would need to either (a) share the private key (unacceptable), (b) re-authenticate (breaks QUIC migration model), or (c) separate transport identity from content identity.
- **Stream state:** If multiple Bitswap streams are active on the connection, migrating mid-transfer requires the preferred peer to understand the Bitswap protocol state for all active streams -- significantly more complex than our current 445-byte migration state.

Feasibility: MEDIUM. The peer identity problem is the fundamental challenge. A solution would require changes to libp2p's authentication model to decouple transport-layer identity from application-layer content verification.

Scenario C: IPFS Cluster Node Failover (Practical)

IPFS Cluster is a tool for orchestrating pinning across multiple IPFS nodes. In this scenario, all nodes in the cluster share a cluster identity and pin the same content. Server-side migration between cluster nodes is highly practical:

- Cluster nodes can share TLS certificates (same organizational identity).
- All cluster nodes pin the same CIDs -- content availability is guaranteed.
- The cluster leader can coordinate migrations based on node health and load.
- This is operationally similar to our current four-machine testbed.

Feasibility: HIGH. IPFS Cluster already has shared identity and content replication. Adding QUIC server migration is a natural extension for zero-downtime node maintenance and load redistribution within a cluster.

2.3 Technical Feasibility Analysis

Aspect	Gateway Migration	P2P Migration	Cluster Failover
TLS Identity	Same cert (gateway operator)	Different PeerIDs (HARD)	Shared cluster cert
Content Availability	Gateway fetches any CID	Must verify via DHT	Same pinset
State Transfer	445 bytes (same as ours)	445 + Bitswap state	445 bytes
QUIC Implementation	quic-go (Go) or quinn (Rust)	Same	Same
Client Modification	None (browser HTTP/3)	libp2p client changes	None
Deployment Control	Centralized	Decentralized (hard)	Semi-centralized
Research Novelty	Low (standard LB)	High (new paradigm)	Medium

2.4 Challenges and Limitations

- **libp2p's QUIC implementation:** libp2p uses quic-go, which does not currently expose the preferred_address transport parameter API. Implementing migration would require modifying quic-go to support preferred_address (similar to how we modified Neqo).
- **Peer authentication model:** libp2p's security model ties the QUIC connection to a specific PeerID. The libp2p TLS handshake extension (draft-ietf-libp2p-tls) embeds the peer's public key in the TLS certificate. Migration to a different peer's key would require a new security model.
- **NAT traversal:** IPFS peers are often behind NATs. The preferred_address must be a publicly reachable address, which may not be available for all peers. IPFS uses hole-punching (DCUtR protocol) for NAT traversal, but this is incompatible with preferred_address which requires a fixed, reachable address.
- **Content verification after migration:** After migration to a new peer, the client should verify it is receiving the correct CID. For Bitswap, this is built-in (every block is verified against its CID hash). For HTTP gateway access, standard HTTP caching mechanisms apply.

2.5 Proposed Experiment: IPFS Gateway Migration PoC

We can demonstrate IPFS gateway migration on our existing testbed with minimal modifications:

1. **Setup:** Run two IPFS gateway instances (Kubo) on opti7040 (primary) and homeserver2 (preferred). Both pin the same test CID.
2. **Modification:** Wrap the IPFS gateway HTTP handler behind our Neqo-based QUIC server. The primary completes the QUIC/TLS handshake, then proxies the HTTP request to the local Kubo gateway API. Upon migration, the preferred server does the same with its local Kubo instance.
3. **Alternative (simpler):** Use our existing migration infrastructure as-is, but serve IPFS content via the HTTP/3 response body. The primary fetches the CID content from its local Kubo node and includes it in the HTTP response. This proves the concept without modifying IPFS internals.
4. **Measurement:** Compare latency of (a) standard IPFS gateway, (b) IPFS gateway behind L4 LB (HAProxy), (c) IPFS gateway with QUIC migration. Measure time-to-first-byte, total transfer time, and migration overhead.

3. Application 2: DNS + Anycast Hybrid Routing

3.1 The Anycast Problem with Stateful Protocols

Anycast is a network addressing technique where the same IP address is advertised from multiple locations via BGP. The network routes each client to the "nearest" server (based on BGP path metrics, not geographic distance). Anycast is widely used for DNS (all 13 root name server addresses are anycast) and CDNs (Cloudflare, Google).

The fundamental problem with anycast and stateful protocols (TCP, QUIC):

- **BGP route flapping:** When BGP routes change (link failure, policy update, route withdrawal), subsequent packets from the same client may be routed to a *different* anycast instance. The new instance has no connection state -- the connection breaks.
- **ECMP hash changes:** Even within a stable anycast setup, ECMP hashing at intermediate routers can shift flows between paths. If paths lead to different anycast instances, connections break.
- **QUIC makes this worse:** QUIC's encrypted headers mean intermediate routers cannot even inspect Connection IDs for consistent routing. Only the initial packet's 5-tuple is reliably usable for ECMP hashing.
- **Current mitigation:** Anycast operators use long BGP hold timers, prepend paths to reduce flapping, and accept that some connections will break. Google's Espresso system uses software-defined anycast with fine-grained traffic engineering to minimize disruption.

3.2 How Server Migration Solves It: The Anycast+Migration Architecture

QUIC server-side migration enables a powerful two-phase routing architecture that combines the simplicity of anycast with the reliability of direct connections:

Phase 1: Anycast-Routed Handshake (Fast)

1. Client sends QUIC Initial packet to anycast VIP (e.g., 192.0.2.1).
2. BGP routes the packet to the nearest anycast PoP (Point of Presence).
3. The PoP server completes the TLS 1.3 handshake (1-RTT or 0-RTT).
4. The handshake is fast because the PoP is geographically close to the client.
5. During the handshake, the server includes `preferred_address` in its transport parameters, pointing to the **unicast IP** of the optimal backend server.

Phase 2: Unicast-Routed Data Transfer (Reliable)

1. The client validates the `preferred_address` via `PATH_CHALLENGE/PATH_RESPONSE`.
2. After validation, the client switches to the unicast address of the backend server.
3. All subsequent data flows directly between client and backend -- no anycast routing.
4. BGP route changes no longer affect the connection (it's on a stable unicast path).
5. The backend may be in a different region than the PoP (e.g., where the client's data lives).

Key insight: Anycast is used only for the initial handshake (1 RTT). The connection then migrates to a stable unicast address, immune to BGP changes. This gives the best of both worlds: low-latency connection establishment AND stable long-lived connections.

DNS + Anycast Combined Flow

The full flow combining DNS and anycast with server migration:

1. **DNS Resolution:** Client resolves example.com. DNS returns an anycast VIP based on geographic proximity (GeoDNS or anycast DNS itself). This provides coarse-grained routing.
2. **Anycast Handshake:** Client connects to the anycast VIP. BGP routes to nearest PoP. PoP server selects the optimal backend based on: (a) client's actual IP (more precise than DNS-based geolocation), (b) current backend load, (c) data locality.
3. **QUIC Migration:** PoP server sends preferred_address pointing to the selected backend's unicast IP. The 445-byte migration state is transferred to the backend.
4. **Direct Connection:** Client migrates to backend. All subsequent traffic bypasses both DNS and anycast routing. Connection is stable for its entire lifetime.

This architecture is particularly valuable for: (a) long-lived connections (WebSocket, gRPC streaming, real-time collaboration), (b) mobile clients whose apparent location may differ from optimal server, and (c) services where data locality matters (database access, personalized content).

3.3 Proposed Experiment: Simulated Anycast + Migration

While we cannot deploy real BGP anycast on our LAN, we can simulate the architecture:

1. **DNS simulation:** Run a local DNS server (dnsmasq) that returns different IPs for the same domain based on client source IP. This simulates GeoDNS.
2. **Anycast simulation:** Use iptables DNAT rules to redirect traffic from a shared VIP to the primary server. This simulates anycast routing to the nearest PoP.
3. **Migration:** Primary server (simulated PoP) completes handshake and migrates to preferred server (simulated backend). Use our existing infrastructure.
4. **Route flap simulation:** Mid-connection, change the iptables DNAT rule to point the VIP to a different server. Show that the migrated connection survives because it's on a unicast path, while a non-migrated connection breaks.
5. **Measurement:** (a) Connection survival rate under simulated BGP flaps, (b) latency overhead of the migration phase, (c) comparison with static unicast assignment.

4. Application 3: QUIC-Native Load Balancing (L3/L4/L7)

4.1 Traditional Load Balancing Layers

Load balancers operate at different network layers, each with distinct capabilities and limitations. Understanding where QUIC server migration fits requires comparing across all three major layers:

Aspect	L3 (Network)	L4 (Transport)	L7 (Application)
Inspects	IP headers only	IP + TCP/UDP ports	Full HTTP request (URL, headers, cookies)
Decision Basis	Dest IP, ECMP hash	5-tuple hash or conn table	Request content, session affinity
State	Stateless (per-packet)	Per-connection (flow table)	Per-request or per-session
Throughput	Line-rate (hardware)	10-40M pps (software)	100K-1M rps (software)
Can Route QUIC?	Yes (IP-level)	Partial (encrypted headers)	Requires TLS termination
Examples	ECMP, Anycast, BGP	Maglev, Katran, IPVS	NGINX, HAProxy, Envoy
QUIC Challenge	No connection awareness	Cannot read QUIC CIDs (encrypted after handshake)	Must terminate QUIC, breaking end-to-end encryption

4.2 QUIC Migration as a Load Balancer

QUIC server-side migration introduces a fundamentally new load balancing paradigm that doesn't fit cleanly into any existing layer. We call it **L4.5 Server-Initiated Load Balancing**:

How It Works as a Load Balancer

1. **Frontend tier (thin):** A set of frontend servers (or a single entry point) handles QUIC handshakes. These are lightweight -- they only need to complete the TLS handshake and decide which backend should serve this connection.
2. **Backend selection:** The frontend selects a backend based on any policy: round-robin, least-connections, consistent hashing, resource-aware (CPU/memory/GPU utilization), or even application-level logic (user affinity, data locality).
3. **State transfer:** The frontend exports the 445-byte migration state (TLS secrets, CIDs) and sends it to the selected backend via any of our five backends (TCP, HTTP, Redis KV, Redis Pub/Sub, File).
4. **Transparent migration:** The client validates the new path and switches to the backend. From this point, all traffic flows directly client-to-backend. The frontend is completely out of the data path.
5. **Post-migration:** The frontend can handle new handshakes immediately. It never becomes a bottleneck because it only processes the handshake (1 RTT) per connection.

Advantages Over Traditional Load Balancers

- **No hairpinning:** Traditional L4/L7 LBs sit in the data path for the entire connection lifetime. Every packet goes Client -> LB -> Backend -> LB -> Client. With QUIC migration, post-handshake traffic is direct: Client

-> Backend. This eliminates the LB as a bandwidth bottleneck and reduces latency.

- **No TLS termination at the LB:** L7 LBs (NGINX, Envoy) must terminate TLS to inspect HTTP headers, breaking end-to-end encryption. QUIC migration preserves the original TLS session -- the LB never sees plaintext application data.
- **Elastic scaling:** The frontend only needs to handle handshake load (1 RTT per conn), not data transfer load. A single frontend can dispatch connections to hundreds of backends. Adding backends requires no LB reconfiguration -- just add the new backend's address to the selection pool.
- **No connection table:** Unlike Maglev (stateful L4 LB), the frontend doesn't need to maintain a per-connection flow table after migration. This makes it immune to state exhaustion attacks (SYN floods).
- **Application-aware routing without L7 overhead:** The frontend can inspect the initial HTTP/3 request (since it terminates TLS for the handshake) and make L7-quality routing decisions, but then get out of the data path. This gives L7 intelligence with L4 efficiency.

Limitations

- **One-shot migration:** RFC 9000 allows only one preferred_address per handshake. Once a connection is migrated to a backend, it cannot be migrated again to another backend using this mechanism. Re-balancing requires connection termination and re-establishment.
- **Migration latency:** The migration adds one PATH_CHALLENGE/PATH_RESPONSE round trip (~1 RTT) between the client and the preferred server. For LAN deployments this is sub-millisecond; for WAN it could be 10-100ms.
- **Client support:** Not all QUIC clients support preferred_address. Firefox does (confirmed in our tests), but Chrome's QUIC implementation (Chromium QUIC) currently ignores preferred_address.
- **No mid-connection rebalancing:** If a backend becomes overloaded after migration, the LB cannot re-migrate that connection. Traditional L4 LBs can at least rate-limit at the LB level.

4.3 Comparison with Existing Approaches

Feature	Maglev (L4)	Envoy (L7)	QUIC-LB	QUIC Migration LB
LB in data path?	Yes (always)	Yes (always)	Yes (always)	No (handshake only)
TLS termination?	No (pass-through)	Yes (re-encrypt)	No	Handshake only
Sees plaintext?	No	Yes	No	Handshake only
Per-conn state at LB?	Yes (flow table)	Yes (session)	No (CID-based)	No (after migration)
Backend selection	At SYN time	Per-request	At Initial	At handshake, with L7 info
Can move backend?	No	Per-request only	No	Yes (once)
Bandwidth bottleneck?	LB throughput	LB throughput	LB throughput	None (direct path)
Client changes?	None	None	CID encoding	None (RFC 9000 standard)

QUIC server migration acts as a "fire-and-forget" load balancer: it makes one high-quality routing decision during the handshake, then steps out of the data path entirely. This is similar to DNS-based load balancing (which also steps out of the data path after resolution) but with much finer granularity and without DNS caching issues.

4.4 Proposed Experiment: Migration Load Balancer PoC

1. **Setup:** Primary server (opti7040) acts as the "LB frontend." Preferred server (homeserver2) acts as "backend." We add a second preferred server (or reuse Redis VM) as a second backend.
2. **Load balancing policy:** Implement round-robin or least-connections selection on the primary. The primary selects which backend to migrate each new connection to.
3. **Workload:** Generate multiple concurrent HTTP/3 connections from the client machine using wrk or h2load with HTTP/3 support. Each connection requests a CPU-intensive response (e.g., compute Pi to N digits).
4. **Measurements:** (a) Request distribution across backends (verify balancing), (b) throughput vs. single-server baseline, (c) migration overhead per connection, (d) frontend CPU utilization (should be minimal after handshakes).
5. **Comparison:** Run the same workload through HAProxy (L4 mode) and HAProxy (L7 mode). Compare bandwidth at the LB, end-to-end latency, and maximum concurrent connections.

5. Application 4: Health-Checked Migration

5.1 The Problem: Migrating to a Dead Server

In our current implementation, the primary server unconditionally includes the `preferred_address` in its transport parameters during the TLS handshake. If the preferred server is unreachable (crashed, network partition, overloaded), the client will attempt `PATH_CHALLENGE` to the preferred address, receive no `PATH_RESPONSE`, and eventually fall back to the primary. This causes:

- **User-visible latency:** The client waits for `PATH_CHALLENGE` timeout before falling back to the primary. RFC 9000 specifies a probe timeout (PTO) which can be several hundred milliseconds to seconds.
- **Wasted resources:** The primary exported and transmitted the migration state (445 bytes) for nothing. Under high load, this wastes CPU cycles on serialization and network bandwidth.
- **Connection instability:** During the probe period, the client may attempt to use the preferred address for data, leading to packet loss and retransmissions.
- **No graceful degradation:** There is no mechanism to inform the client that migration failed -- the client must discover this through timeout.

5.2 Health Check Design

We propose a **pre-migration health check** mechanism where the primary server verifies the preferred server's availability before including `preferred_address` in the handshake. Three approaches, ordered by increasing sophistication:

Approach 1: Passive Health Check (Heartbeat)

The preferred server periodically sends heartbeat messages to the primary (or to a shared coordination service like Redis). The primary only includes `preferred_address` if a heartbeat was received within the last N seconds.

- **Implementation:** Preferred server publishes a heartbeat to Redis every 1 second (`SET preferred:health "alive" EX 3`). Primary checks this key before each handshake.
- **Pros:** Simple, low overhead, works with all state transfer backends.
- **Cons:** Stale information -- preferred could crash between heartbeat and migration. Detection window is bounded by heartbeat interval.
- **Latency added to handshake:** One Redis GET (~0.1ms on LAN).

Approach 2: Active Health Check (Probe)

The primary server actively probes the preferred server before each handshake (or periodically). The probe checks both network reachability and application readiness.

- **Implementation:** Primary sends a lightweight UDP probe to the preferred server's migration port (9999). The preferred responds with its current capacity (available memory, CPU load, active connections). Primary only migrates if capacity is sufficient.
- **Pros:** Fresh, real-time health information. Can include load metrics for intelligent routing decisions.
- **Cons:** Adds latency to the handshake path (1 RTT to preferred). Probe itself can fail (timeout). Under high connection rates, probe traffic becomes significant.
- **Latency added to handshake:** 1 RTT to preferred server (~0.2ms on LAN, 10-100ms WAN).

Approach 3: State Transfer as Health Check (Integrated)

Use the state transfer mechanism itself as the health check. The primary attempts to send the migration state to the preferred server. If the transfer succeeds, migration proceeds; if it fails (connection refused, timeout), the primary serves the connection directly without `preferred_address`.

- **Implementation:** For TCP Push backend: if `connect()` to `preferred:9999` fails, skip migration. For Redis KV: if SET succeeds and preferred ACKs (via a response key), proceed. For HTTP Pull: primary stores state and waits for preferred to pull it; if not pulled within timeout, serve directly.
- **Pros:** No separate health check mechanism -- the state transfer IS the health check. Zero additional overhead when migration succeeds. Naturally handles partial failures.
- **Cons:** For Immediate (eager) transfer, the state is sent during the handshake, so the handshake blocks on the transfer. For Lazy transfer, the state isn't sent until the client connects to the preferred, so the health check happens too late.
- **Best with:** TCP Push + short timeout (50ms). If the TCP connection to `preferred:9999` isn't established within 50ms, skip migration and serve directly.

Recommendation: Start with Approach 1 (passive heartbeat via Redis) for simplicity, then upgrade to Approach 3 (integrated state-transfer-as-health-check) for production. Approach 2 adds too much latency for the marginal benefit over Approach 1.

5.3 Implementation Plan

Adding health checks to our existing Neqo implementation requires changes in two places:

1. **Primary server (`primary_server.rs`):** Before calling `connection.set_preferred_address()`, check preferred health. If unhealthy, skip the `preferred_address` transport parameter entirely. The connection completes normally without migration.
2. **Preferred server (`preferred_server.rs`):** Add a heartbeat thread that periodically writes to Redis (or responds to UDP probes). Also report capacity metrics: current connection count, CPU load, available memory.
3. **Graceful fallback:** If the primary decides not to migrate (preferred unhealthy), it serves the HTTP/3 response directly. The client never knows migration was considered.
4. **Monitoring:** Add counters for: migrations attempted, migrations succeeded, migrations skipped (unhealthy), migrations failed (preferred died after health check passed).

5.4 Proposed Experiment: Health Check Validation

1. **Baseline:** Run 100 sequential connections with preferred server healthy. Measure migration success rate (should be 100%).
2. **Failure injection:** Kill the preferred server process mid-experiment. Without health checks: measure how many connections experience timeouts. With health checks: measure how many connections are gracefully served by the primary instead.
3. **Flapping:** Start/stop the preferred server every 5 seconds. Compare user-visible error rates with and without health checks.
4. **Latency comparison:** Measure handshake latency with each health check approach (heartbeat, probe, integrated) vs. no health check.
5. **Metric:** The key metric is **zero failed migrations** -- every connection should either migrate successfully or be served directly by the primary. No connection should experience a timeout due to migrating to a dead server.

6. Application 5: BitTorrent Peer Migration

6.1 BitTorrent Architecture and Transport

BitTorrent is the most widely deployed peer-to-peer file sharing protocol, responsible for an estimated 2-5% of all Internet traffic. Unlike IPFS, which adopted QUIC as its default transport in 2023, BitTorrent still relies on TCP and uTP (Micro Transport Protocol) for peer-to-peer data transfer. Key architectural properties relevant to server migration:

- **Piece-based content verification:** Files are divided into fixed-size pieces (typically 256KB-4MB). Each piece has a SHA-1 hash (v1) or SHA-256 Merkle tree hash (v2, BEP 52). Clients verify every piece against its hash, guaranteeing content integrity regardless of which peer served it. This is directly analogous to IPFS's CID-based verification.
- **Swarm model:** A client (leecher) connects to many peers simultaneously, downloading different pieces from different seeders. Losing a single connection does not halt the download -- the client simply requests those pieces from other peers. However, reconnection incurs latency (tracker/DHT re-query + new handshake).
- **Transport layer:** BitTorrent uses TCP for the main peer protocol (BEP 3) and uTP (BEP 29) as a UDP-based congestion-controlled alternative. Neither supports connection migration. There is **no existing BEP for QUIC transport**, making this a greenfield research area.
- **WebSeed (BEP 19):** Torrents can specify HTTP/FTP URLs as additional download sources. Clients download pieces from web servers using HTTP range requests. As web servers adopt HTTP/3 (QUIC), WebSeeds become a natural entry point for QUIC migration.
- **Tracker and DHT:** Peers discover each other via centralized trackers (BEP 3) or the distributed hash table (BEP 5, Mainline DHT). When a peer disconnects, the client must re-query these to find replacement peers -- a process that takes 1-10 seconds depending on DHT latency and tracker responsiveness.

6.2 How QUIC Server Migration Applies

The core insight is that BitTorrent shares two critical properties with IPFS that make server-side migration viable: **content-addressed data** (piece hashes guarantee integrity) and **multiple providers** (many peers hold the same pieces). A seeder serving pieces over QUIC could use the `preferred_address` mechanism to redirect the client to another seeder that holds the same pieces, achieving seamless failover without tracker/DHT re-query.

Crucially, BitTorrent has an advantage over IPFS for migration: peer identity is NOT cryptographically bound to the TLS certificate. In IPFS, libp2p embeds the PeerID in the TLS certificate, making migration between different peers fundamentally difficult. BitTorrent peers are identified by a 20-byte PeerID that is independent of the transport layer, making TLS state transfer between peers straightforward.

6.3 Three Migration Scenarios

Scenario A: WebSeed Migration via HTTP/3 (HIGH Feasibility)

WebSeeds (BEP 19) allow torrent clients to download pieces from standard HTTP servers. As CDNs and web servers adopt HTTP/3, WebSeeds will increasingly use QUIC. A WebSeed server can use `preferred_address` to migrate the client to a mirror server that hosts the same files.

- **How it works:** Client connects to WebSeed A over HTTP/3. WebSeed A includes `preferred_address` pointing to WebSeed B (a mirror). After handshake, client migrates to WebSeed B. Client continues

downloading pieces via HTTP range requests -- identical to our current implementation.

- **Directly analogous:** This is functionally identical to IPFS gateway migration. WebSeed A = primary server, WebSeed B = preferred server. Both serve the same file via HTTP.
- **No client modification:** Any HTTP/3 client (browser, curl, aria2) handles the migration transparently via standard QUIC `preferred_address`.
- **Use case:** CDN-backed torrent distribution (e.g., Linux distro mirrors) where multiple HTTP servers host the same .iso file.

Scenario B: Seeder Failover (MEDIUM Feasibility)

A seeder planning to go offline (maintenance, churn) can proactively migrate its connected leechers to another seeder that holds the same pieces. This avoids the costly tracker/DHT re-query and new QUIC handshake.

- **How it works:** Seeder A detects it will go offline (graceful shutdown signal, scheduled maintenance, battery low on mobile). It queries the tracker/DHT for other seeders with overlapping piece sets, selects one (Seeder B), and announces `preferred_address` pointing to Seeder B. Migration state (445 bytes) is transferred to Seeder B via any of our five backends.
- **Piece availability check:** Before migrating, Seeder A must verify that Seeder B has the pieces the leecher needs. This can be done by exchanging `BITFIELD` messages (standard BT protocol) or consulting the tracker's peer metadata.
- **Cost savings:** Without migration, recovery takes: DHT re-query (~1-10s) + new QUIC handshake (1 RTT) + piece re-negotiation. With migration: just 1 RTT (`PATH_CHALLENGE/PATH_RESPONSE`). For large swarms with frequent churn, this saves thousands of reconnections per hour.
- **Requires:** A QUIC-based BitTorrent peer protocol. This does not exist today and would need a new BEP proposal.

Scenario C: Tracker-Coordinated Load Distribution (NOVEL)

A heavily loaded seeder can use the tracker as a coordination point to migrate leechers to less-loaded seeders. This is a novel form of **peer-assisted load balancing** that has no equivalent in today's BitTorrent ecosystem.

- **How it works:** Seeder A is serving 100 leechers and approaching its upload bandwidth limit. It queries the tracker for other seeders of the same torrent, finds Seeders B, C, D with lower load. Seeder A migrates 30 leechers to each via `preferred_address`.
- **Tracker as coordinator:** The tracker already knows which peers have which torrents. Adding load metrics (current connections, upload bandwidth used) to tracker announce messages enables intelligent migration decisions.
- **Novel contribution:** This is fundamentally different from existing BitTorrent load management (which relies on choking/unchoking algorithms). Migration-based load distribution moves entire connections rather than throttling individual peers.
- **Research potential:** This could significantly improve piece distribution in swarms with uneven seeder load. The tracker could implement global optimization strategies (e.g., minimize total download time across all leechers).

6.4 Comparison with IPFS Migration

BitTorrent and IPFS share similar properties for migration, but differ in key aspects:

Aspect	IPFS	BitTorrent
Content verification	CID hash (SHA-256)	Piece hash (SHA-1/SHA-256)
Multiple providers	Yes (DHT providers)	Yes (swarm seeders)
QUIC adoption	Default since Kubo 0.18+ (2023)	Not yet -- uses TCP/uTP
Identity bound to TLS	Yes (PeerID = hard to migrate)	No (PeerID independent of transport -- easier)
Connection multiplexing	Many streams per conn (Bitswap, DHT, etc.)	One torrent per conn (simpler state)
Best migration scenario	Gateway (HTTP/3)	WebSeed (HTTP/3)
State beyond TLS	Bitswap streams, DHT queries (complex)	Piece bitmap, request queue (simpler)
Research novelty	High (no prior work)	Very high (no prior work, no QUIC in BT at all)

6.5 Challenges

- **No QUIC transport for BitTorrent:** The biggest barrier. BitTorrent uses TCP and uTP; there is no BEP for QUIC transport. WebSeed via HTTP/3 is the only path that works today without protocol changes. For native peer-to-peer migration (Scenarios B and C), a QUIC-based peer protocol would need to be designed.
- **Piece availability verification:** Before migrating a leecher to another seeder, the seeder must verify that the target has the needed pieces. Full seeders (100% of pieces) are trivially compatible, but partial seeders require bitfield comparison. This adds a coordination step not present in our current implementation.
- **In-flight piece requests:** If the leecher has outstanding piece requests to the migrating seeder, those requests must be re-issued to the new seeder. Unlike IPFS (where Bitswap streams carry complex state), BitTorrent piece requests are stateless -- the leecher simply re-sends REQUEST messages after migration.
- **Swarm dynamics:** BitTorrent's tit-for-tat algorithm (choking/unchoking) is designed around direct reciprocity. A migrated leecher has no upload history with the new seeder, which may affect its initial unchoke priority. However, optimistic unchoking (standard in all BT clients) ensures new connections eventually get service.
- **NAT traversal:** Many BitTorrent peers are behind NATs. QUIC's preferred_address mechanism requires the preferred server to be directly reachable, which may not be true for residential peers. This limits Scenarios B and C to peers with public IPs or relay-assisted setups.

Key insight: WebSeed migration (Scenario A) sidesteps all peer-to-peer challenges. Web servers have public IPs, serve complete files, and already use HTTP/3. This is the recommended starting point -- identical to our current implementation.

6.6 Proposed Experiment: WebSeed Migration PoC

1. **Setup:** Deploy two HTTP/3 web servers (opti7040 and homeserver2) serving the same large file (e.g., a 1GB Linux ISO). Create a .torrent file with both servers listed as WebSeeds (url-list). Configure opti7040 as

the primary with preferred_address pointing to homeserver2.

2. **Client:** Use aria2 (supports WebSeed + HTTP/3) or a modified qBittorrent to download the torrent. Alternatively, use Firefox to download directly via HTTP/3 (testing the WebSeed URL directly, not the torrent protocol).
3. **Metric 1 -- Seamless failover:** Start downloading from opti7040, trigger migration mid-download. Verify the download completes successfully from homeserver2 without restart.
4. **Metric 2 -- Latency comparison:** Compare recovery time when the primary WebSeed fails: (a) without migration (client must discover and connect to backup WebSeed), vs. (b) with migration (preferred_address, 1 RTT failover).
5. **Metric 3 -- Piece integrity:** Verify all downloaded pieces pass SHA-1/SHA-256 verification, confirming that migration did not corrupt the data stream.
6. **Extension:** If time permits, prototype a QUIC-based BitTorrent peer protocol using Neqo, demonstrating native peer-to-peer migration (Scenario B) between two seeders.

7. Comparative Analysis of All Applications

The following table compares all five applications across key research dimensions:

Dimension	IPFS	DNS+Anycast	Load Balancing	Health Checks	BitTorrent
Research Novelty	HIGH -- new paradigm for content nets	MEDIUM -- known idea, new mechanism	MEDIUM -- novel approach to known problem	LOW -- engineering contribution	VERY HIGH -- no QUIC in BT, unexplored
Impl. Complexity	HIGH -- modify libp2p or wrap gateway	MEDIUM -- simulate anycast on LAN	LOW -- extend existing primary server	LOW -- add health check to primary	LOW (WebSeed) to HIGH (native peer)
Uses Existing Testbed?	Partially -- need IPFS nodes	Yes -- with iptables simulation	Yes -- directly	Yes -- directly	Yes -- WebSeed uses existing HTTP/3
Time to PoC	2-3 weeks	1-2 weeks	1 week	3-5 days	1 week (WebSeed) 3-4 weeks (native)
Paper Potential	Standalone paper (top venue)	Section in systems paper	Section in systems paper	Engineering section in existing paper	Standalone paper (top venue)
Professor Appeal	HIGH -- novel, interdisciplinary	HIGH -- practical, industry-relevant	MEDIUM -- well-studied area	HIGH -- shows rigor and completeness	HIGH -- large user base, novel angle

Scoring Summary

Application	Novelty (1-5)	Feasibility (1-5)	Impact (1-5)	Effort (1-5, low=good)	Total (out of 20)
IPFS Gateway Migration	5	4	4	3	16
DNS + Anycast Hybrid	4	3	5	3	15
QUIC Migration LB	3	5	4	4	16
Health-Checked Migration	2	5	3	5	15

Application		Novelty (1-5)	Feasibility (1-5)	Impact (1-5)	Effort (1-5, low=good)	Total (out of 20)
BitTorrent Migration	WebSeed	5	4	5	4	18

8. Recommended Research Roadmap

Based on the analysis above, we recommend the following phased approach:

Phase 1: Health-Checked Migration (Immediate)

Add health checks to the existing implementation. This is the lowest-hanging fruit and strengthens any subsequent experiments by ensuring robust migration behavior. Implement Redis-based heartbeat (Approach 1) first, then integrate health checking into the state transfer mechanism (Approach 3).

Deliverable: Modified `primary_server.rs` with health-aware migration decision. Experiment results showing zero failed migrations under server failure.

Phase 2: QUIC Migration Load Balancer (Short-term)

Extend the primary server to act as a load balancer frontend. Implement backend selection policies (round-robin, least-connections). Benchmark against HAProxy. This directly builds on the existing implementation and the health check work from Phase 1.

Deliverable: Load balancer PoC with performance comparison. Demonstrates the "LB bypass" use case described in RFC 9000 Section 9.6.

Phase 3: DNS + Anycast Hybrid (Medium-term)

Build the two-phase routing architecture with simulated anycast. Demonstrate connection stability under route changes. This requires the LB infrastructure from Phase 2 and adds the anycast simulation layer.

Deliverable: Anycast+migration PoC showing connection survival under route flaps. Quantified latency comparison with pure anycast.

Phase 4: IPFS Integration (Long-term / Separate Paper)

Implement IPFS gateway migration as a standalone contribution. This could form the basis of a separate research paper on content-addressed network migration. Requires IPFS/Kubo deployment on the testbed and potentially modifications to quic-go or a Rust-native IPFS client.

Deliverable: IPFS gateway migration PoC with performance evaluation. Paper submission to a networking or distributed systems venue.

Phase 5: BitTorrent WebSeed Migration (Parallel with Phase 4)

Implement WebSeed migration using the existing testbed. Deploy two HTTP/3 servers hosting the same large file, create a torrent with both as WebSeeds, and demonstrate seamless failover via `preferred_address`. This reuses our existing Neqo code with minimal changes -- just serve a large file instead of the demo page. Can run

in parallel with IPFS work since it shares the same infrastructure.

Deliverable: WebSeed migration PoC, latency comparison (migration vs. re-discovery), and piece integrity verification. If successful, design a BEP draft for QUIC-based BitTorrent peer protocol with migration support.

9. References

QUIC and Migration

- [1] RFC 9000, "QUIC: A UDP-Based Multiplexed and Secure Transport," IETF 2021. Section 9.6: Server's Preferred Address.
- [2] Grubl et al., "QUIC-Exfil: Exploiting QUIC's Server Preferred Address Feature," ASIA CCS 2025.
- [3] Puliafito et al., "Server-side QUIC connection migration to support microservice deployment at the edge," Pervasive and Mobile Computing, 2022.
- [4] Duke et al., "QUIC-LB: Generating Routable QUIC Connection IDs," IETF Draft, 2025.

IPFS and libp2p

- [5] Benet, J., "IPFS - Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.
- [6] libp2p Specification, "QUIC Transport." <https://github.com/libp2p/specs/tree/master/quic>
- [7] libp2p TLS 1.3 Handshake, "Using TLS 1.3 with libp2p." <https://github.com/libp2p/specs/blob/master/tls/tls.md>
- [8] IPFS Cluster Documentation. <https://ipfcluster.io/>
- [9] Protocol Labs, "Kubo: IPFS Implementation in Go." <https://github.com/ipfs/kubo>

DNS and Anycast

- [10] RFC 4786, "Operation of Anycast Services," IETF 2006.
- [11] Flavel, A. et al., "FastRoute: A Scalable Load-Aware Anycast Routing Architecture for Modern CDNs," NSDI 2015.
- [12] Yap, K.K. et al., "Taking the Edge off with Espresso: Scale, Reliability and Programmability for Global Internet Peering," SIGCOMM 2017.
- [13] Fan, X. et al., "Consistent Hashing with Bounded Loads," STOC 2017.

Load Balancing

- [14] Eisenbud et al., "Maglev: A Fast and Reliable Software Network Load Balancer," NSDI 2016.
- [15] Olteanu et al., "Stateless Datacenter Load-balancing with Beamer," NSDI 2018.
- [16] Miao et al., "SilkRoad: Making Stateful Layer-4 Load Balancing Fast and Cheap," SIGCOMM 2017.
- [17] Barbet et al., "Cheetah: A High-Speed Load-Balancer Design with Guaranteed Per-Connection-Consistency," NSDI 2020.
- [18] Facebook Engineering, "Open-sourcing Katran," 2018.

Health Checking and Fault Tolerance

- [19] Consul by HashiCorp, "Health Checks." <https://developer.hashicorp.com/consul/docs/services/usage/checks>
- [20] Envoy Proxy, "Health Checking." https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/health_checking
- [21] Kubernetes, "Configure Liveness, Readiness and Startup Probes." <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

BitTorrent and Peer-to-Peer

- [22] Cohen, B., "The BitTorrent Protocol Specification," BEP 3, 2008. https://www.bittorrent.org/beps/bep_0003.html
- [23] BEP 52, "The BitTorrent Protocol Specification v2," 2020. https://www.bittorrent.org/beps/bep_0052.html
- [24] BEP 19, "WebSeed - HTTP/FTP Seeding (GetRight style)," 2008. https://www.bittorrent.org/beps/bep_0019.html
- [25] BEP 17, "HTTP Seeding (Hoffman-style)," 2008. https://www.bittorrent.org/beps/bep_0017.html

- [26] BEP 29, "uTorrent transport protocol (uTP)," 2009. https://www.bittorrent.org/beps/bep_0029.html
- [27] BEP 5, "DHT Protocol," 2008. https://www.bittorrent.org/beps/bep_0005.html
- [28] Norberg, A., "libtorrent: A C++ BitTorrent implementation." <https://www.libtorrent.org/>